LOCA: Authorized LOCO

Nameless Author(s)

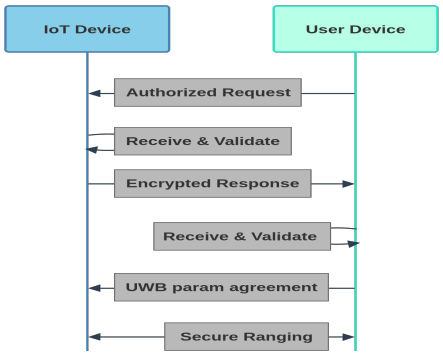

Figure 1. LOCA Runtime Protocol Overview

1. Authorized LOCO: LOCA

LOCA (Authorized LOCO) extends LOCO for private settings such as hotels and vacation rentals through a time-bound authorization mechanism. In these environments, device discovery must be restricted: only property owners and their authorized guests should gain awareness of IoT device presence, capabilities, and locations during their stay **[$\mathcal{F}$1.5]**. Without authorization controls, unauthorized individuals could surveil the space's IoT Infrastructure. Additionally, unrestricted discovery would expose devices from adjacent rooms or other parties, both overwhelming users with irrelevant information from outside the operational boundaries of the rental space.

1.1. Overview

LOCA involves four entities: The property owner ($\mathcal{O}$), user devices ($\mathsf{U_{dev}}$-s) carried by authorized guests, IoT devices ($\mathsf{I_{dev}}$-s) deployed within the space, and the device manufacturer (Mfr).

The protocol operates in three phases: *Registration*, where Mfr provisions devices and $\mathcal{O}$ configures space-specific credentials; *Authorization*, where $\mathcal{O}$ provisions temporary access credentials to authorized $\mathsf{U_{dev}}$; and *Runtime*, where $\mathsf{U_{dev}}$ discovers and localizes $\mathsf{I_{dev}}$-s within the authorized space.

LOCA ***Registration***: $\mathsf{I_{dev}}$ provisioning follows the same process as in LOCO. Additionally, $\mathcal{O}$ registers each rental unit with a unique group key (K_g) shared among all $\mathsf{I_{dev}}$ within that space. K_g is stored securely in each device's TCB.

LOCA ***Authorization***: During the *Authorization* phase, $\mathcal{O}$ provisions temporary credentials to authorized guests. For each guest, $\mathcal{O}$ generates a random nonce (R_u) and defines the guest's stay period as $\mathsf{Stay_T} = (T_s, T_e)$, where T_s and T_e represent the authorization start and end times. $\mathcal{O}$ then derives a guest-specific subkey $Ku = \text{KDF}(R_u, K_g, T_s, T_e)$ and securely transmits both K_u and $\mathsf{Stay_T}$ to $\mathsf{U_{dev}}$ (e.g., via encrypted messaging, TLS, or NFC at check-in).

LOCA ***Runtime***: The *Runtime* phase consists of five steps: (1) $\mathsf{U_{dev}}$ sends an authenticated request ($\mathsf{Msg_{Authreq}}$), (2) $\mathsf{I_{dev}}$ validates the authentication request, derives a session key, and generates UWB ranging parameters, (3) $\mathsf{I_{dev}}$ responds with an encrypted reply ($\mathsf{Msg_{rep}}$) containing device information and UWB parameters, (4) $\mathsf{U_{dev}}$ decrypts and validates the response by verifying the device manifest, and (5) secure ranging determines $\mathsf{I_{dev}}$-s location.

1.2. Adversary Model

$\mathcal{A}$dv remains the same as LOCO with the following additions:

DoS attacks on $\mathsf{I_{dev}}$-s: DoS attacks from a network $\mathcal{A}$dv are more challenging to address in LOCA due to the cost of MAC verification on every $\mathsf{Msg_{Authreq}}$. $\mathcal{A}$dv can flood $\mathsf{I_{dev}}$ with malformed authenticated requests, MAC computations using recomputed guest keys. These computational costs are minimized through design choices justified in Section 1.4.2.

Replay Attacks: A network-based $\mathcal{A}$dv can replay $\mathsf{Msg_{Authreq}}$ and $\mathsf{Msg_{rep}}$ to both $\mathsf{U_{dev}}$ and $\mathsf{I_{dev}}$-s. However, $\mathsf{Msg_{Authreq}}$ includes a fresh nonce ($\mathcal{N}_{\mathsf{dev}}$) that $\mathsf{I_{dev}}$ validates. Similarly, $\mathsf{Msg_{rep}}$ is cryptographically bound to the ephemeral public key ($\mathsf{pk_{U_{dev}}}$) included in the original $\mathsf{Msg_{Authreq}}$. Each response is session-specific.

Eavesdropping: A network-based $\mathcal{A}$dv can eavesdrop on all LOCA protocol messages. However, since $\mathsf{I_{dev}}$ only responds to properly authenticated requests with valid temporal credentials, $\mathcal{A}$dv must wait for legitimate user activity to observe $\mathsf{Msg_{rep}}$-s containing encrypted device information. From these observations, $\mathcal{A}$dv can infer the number of deployed $\mathsf{I_{dev}}$-'s and attempt to link occurrences to specific $\mathsf{I_{dev}}$-s across multiple discovery sessions.

1.3. LOCA Security Requirements

Security requirements for LOCA are similar to those of LOCO, with the following additions:

- *Request authentication*: $\mathsf{I_{dev}}$ must validate each $\mathsf{Msg_{Authreq}}$ by verifying: (1) temporal validity of $\mathsf{Stay_T}$

Notation	Description
$\mathcal{O}$	Property (IoT space) owner/manager
K_g	Group key unique to each property, shared among all $\mathsf{I_{dev}}$-s in it
K_u	Guest-specific subkey: $K_u = \text{KDF}(R_u, K_g, T_s, T_e)$
R_u	Random nonce unique to each guest, used in K_u derivation
$\mathsf{Stay_T}$	Guest stay period: $\mathsf{Stay_T} = (\mathsf{T_s}, \mathsf{T_e})$
T_s	Guest authorization start time
T_e	Guest uthorization end time
$\mathsf{Msg_{Authreq}}$	Authenticated request message from $\mathsf{U_{dev}}$: $[\mathcal{N}_{\mathsf{dev}}, \mathsf{Stay_T}, R_u, \mathsf{pk_{U_{dev}}}, \text{MAC}_{K_u}(\mathcal{N}_{\mathsf{dev}}, \mathsf{Stay_T}, R_u, \mathsf{pk_{U_{dev}}})]$
$\mathsf{Msg_{rep}}$	Secure reply from $\mathsf{I_{dev}}$: $[\mathcal{N}_{\mathsf{dev}}, \mathsf{pk_{I_{dev}}}, \mathcal{ENC}_{\mathsf{Msg_{rep}}}, \mathsf{IV_{AES}}, \mathcal{SIG}_{\mathsf{Msg_{rep}}}]$
$\mathcal{SIG}_{\mathsf{Msg_{rep}}}$	Signature in $\mathsf{Msg_{rep}}$: $\mathcal{SIG}_{\mathsf{sk_{I_{dev}}}}(\mathcal{N}_{\mathsf{dev}}, \mathsf{pk_{I_{dev}}}, \mathcal{ENC}_{\mathsf{Msg_{rep}}}, \mathsf{IV_{AES}})$
$\mathcal{ENC}_{\mathsf{Msg_{rep}}}$	Encrypted $\mathsf{Msg_{rep}}$ using $\mathcal{K}_{\mathsf{I_{dev}},\mathsf{U_{dev}}}$: $[\mathcal{ENC}(\mathsf{URL_{Man}}, \mathcal{A}\mathsf{ttest_{result}}, \mathcal{K}_{STS}, \mathsf{IV_{STS}}, \mathsf{Src_{UWB}}, \mathsf{Dst_{UWB}}), \mathsf{Tag_{AES}}]$

TABLE 1. LOCA NOTATION SUMMARY

against current time, (2) freshness of $\mathcal{N}_{\mathsf{dev}}$, and (3) MAC authenticity using the recomputed K_u.

- *Response confidentiality*: Device information ($\mathsf{URL_{Man}}$, $\mathcal{A}\mathsf{ttest_{result}}$) and UWB ranging parameters ($\mathcal{K}_{STS}$, $\mathsf{IV_{STS}}$, $\mathsf{Src_{UWB}}$, $\mathsf{Dst_{UWB}}$) in $\mathsf{Msg_{rep}}$ must remain confidential to unauthorized parties.

Modified requirements from LOCO:

- *Timeliness:* Unlike LOCO, LOCA does not require periodic announcements. Instead, $\mathsf{I_{dev}}$ must respond promptly to authenticated requests, ensuring timely device discovery for authorized $\mathsf{U_{dev}}$.

1.4. Protocol Design

1.4.1. Registration. $\mathsf{I_{dev}}$ provisioning follows the same process as in LOCO. Additionally, $\mathcal{O}$ registers each rental unit with a unique group key (K_g) shared among all $\mathsf{I_{dev}}$ within that space. K_g is stored securely in each device's TCB.

TCB differences from LOCO: Unlike LOCO, LOCA operates on a request-response model rather than periodic broadcasts. Consequently, the secure timer peripheral is not required in LOCA's TCB, as device announcements are only transmitted in response to authenticated requests from authorized $\mathsf{U_{dev}}$.

1.4.2. Authorization. The *Authorization* phase addresses the challenge of authenticating guests' device discovery requests in private spaces. Several approaches exist for implementing time-bound authorization:

A Kerberos-based approach, where guests obtain tickets from a centralized authentication server, provides strong security guarantees but requires deploying and maintaining Kerberos infrastructure. Each rental property would need a dedicated Key Distribution Center, and all devices must be integrated into the Kerberos realm. Such an overhead is impractical for typical hotel or rental deployments.

An alternative approach involves $\mathcal{O}$ issuing signed certificates to each guest, containing their ephemeral public key and validity period. Each $\mathsf{I_{dev}}$ would then store $\mathcal{O}$'s public key in its TCB and verify certificate signatures on discovery requests. While conceptually sound, this introduces computational overhead: every discovery attempt requires signature verification on resource-constrained devices.

A hybrid approach combines certificates with symmetric authentication: certificates contain a guest-specific nonce R_u, and guests authenticate using K_u. However, certificate verification overhead remains, albeit with more efficient MAC operations for request authentication.

LOCA adopts a certificate-free approach that eliminates public-key operations from the request authentication path. $\mathcal{O}$ directly provisions guests with derived keys, leveraging efficient MAC verification on devices. While this requires secure out-of-band key distribution, such channels (encrypted messaging, NFC at check-in) are already standard in modern hospitality workflows.

Authorization Protocol: During check-in, $\mathcal{O}$ performs the following for each guest:

1) Generates a random nonce R_u.
2) Defines guest's authorization period: $\mathsf{Stay_T} = (T_s, T_e)$.
3) Derives a guest-specific subkey K_u per Table 1.
4) Securely transmits K_u and $\mathsf{Stay_T}$ to $\mathsf{U_{dev}}$.

K_u is cryptographically bound to both the space-specific K_g and the time-limited $\mathsf{Stay_T}$.

1.4.3. Runtime. The *Runtime* phase is similar to that of LOCO with additional steps for guest authentication and device information confidentiality.

Authenticated Request: $\mathsf{U_{dev}}$ initiates by broadcasting $\mathsf{Msg_{Authreq}}$, which includes fresh nonce ($\mathcal{N}_{\mathsf{dev}}$), the guest's stay period ($\mathsf{Stay_T}$), the unique guest identifier (R_u), $\mathsf{U_{dev}}$'s ephemeral public key ($\mathsf{pk_{U_{dev}}}$), and a MAC computed over the request contents using K_u:

$$\mathsf{Msg_{Authreq}} = [\mathcal{N}_{\mathsf{dev}}, \mathsf{Stay_T}, \mathsf{R_u}, \mathsf{pk_{U_{dev}}}, \text{MAC}_{\mathsf{K_u}}(\mathcal{N}_{\mathsf{dev}}, \mathsf{Stay_T}, \mathsf{R_u}, \mathsf{pk_{U_{dev}}})]$$

Reception and Validation on $\mathsf{I_{dev}}$: Upon receiving $\mathsf{Msg_{Authreq}}$, $\mathsf{I_{dev}}$ validates the request by verifying that the current time falls within the authorization window ($\mathsf{Stay_T}$) and that $\mathcal{N}_{\mathsf{dev}}$ is fresh. $\mathsf{I_{dev}}$ then recomputes the guest-specific subkey $K_u = \text{KDF}(R_u, K_g, \mathsf{Stay_T})$ and verifies the MAC using the recomputed K_u. If validation succeeds, $\mathsf{I_{dev}}$ proceeds to prepare an encrypted response.

Session Key Derivation and UWB Parameter Generation: $\mathsf{I_{dev}}$ derives a session-specific shared secret: $\mathcal{K}_{\mathsf{I_{dev}},\mathsf{U_{dev}}} = \text{ECDH}(\mathsf{sk_{I_{dev}}}, \mathsf{pk_{U_{dev}}})$. $\mathsf{I_{dev}}$ then generates UWB ranging parameters: $\mathcal{K}_{STS}$, $\mathsf{IV_{STS}}$, $\mathsf{Src_{UWB}}$, and $\mathsf{Dst_{UWB}}$. These parameters, along with device information ($\mathsf{URL_{Man}}$, $\mathcal{A}\mathsf{ttest_{result}}$), are encrypted using $\mathcal{K}_{\mathsf{I_{dev}},\mathsf{U_{dev}}}$:

$$\mathcal{ENC}_{\mathsf{Msg_{rep}}} = [\mathcal{ENC}_{\mathcal{K}_{\mathsf{I_{dev}},\mathsf{U_{dev}}}}(\mathsf{URL_{Man}}, \mathcal{A}\mathsf{ttest_{result}}, \mathcal{K}_{STS}, \mathsf{IV_{STS}}, \mathsf{Src_{UWB}}, \mathsf{Dst_{UWB}}), \mathsf{Tag_{AES}}]$$

Encrypted Response from $\mathsf{I_{dev}}$: $\mathsf{I_{dev}}$ constructs and broadcasts $\mathsf{Msg_{rep}}$:

$$\mathcal{SIG}_{\mathsf{Msg_{rep}}} = \mathcal{SIG}_{\mathsf{sk_{I_{dev}}}}(\mathcal{N}_{\mathsf{dev}}, \mathsf{pk_{I_{dev}}}, \mathcal{ENC}_{\mathsf{Msg_{rep}}}, \mathsf{IV_{AES}})$$

$$\mathsf{Msg_{rep}} = [\mathcal{N}_{\mathsf{dev}}, \mathsf{pk_{I_{dev}}}, \mathcal{ENC}_{\mathsf{Msg_{rep}}}, \mathsf{IV_{AES}}, \mathcal{SIG}_{\mathsf{Msg_{rep}}}]$$

where $\mathsf{pk_{I_{dev}}}$is $\mathsf{I_{dev}}$'s long-term public key (matching the key in its manifest). After broadcasting $\mathsf{Msg_{rep}}$, $\mathsf{I_{dev}}$ configures its UWB module with the generated ranging parameters and passively awaits ranging initiation from $\mathsf{U_{dev}}$.

Reception and Validation on $\mathsf{U_{dev}}$: Upon receiving $\mathsf{Msg_{rep}}$, $\mathsf{U_{dev}}$ extracts $\mathsf{pk_{I_{dev}}}$and derives the shared secret $\mathcal{K}_{\mathsf{I_{dev}},\mathsf{U_{dev}}}$ = ECDH($\mathsf{sk_{U_{dev}}}$, $\mathsf{pk_{I_{dev}}}$). Using $\mathcal{K}_{\mathsf{I_{dev}},\mathsf{U_{dev}}}$, $\mathsf{U_{dev}}$ decrypts $\mathcal{E}\mathsf{NC}_{\mathsf{Msg_{rep}}}$to obtain device information ($\mathsf{URL_{Man}}$, $\mathcal{A}\mathsf{ttest}_{\mathsf{result}}$) and UWB ranging parameters ($\mathcal{K}_{STS}$, $\mathsf{IV_{STS}}$, $\mathsf{Src_{UWB}}$, $\mathsf{Dst_{UWB}}$). $\mathsf{U_{dev}}$ then fetches $\mathsf{Manifest_{I_{dev}}}$ from $\mathsf{URL_{Man}}$ and performs cryptographic validation: first verifying Mfr's signature on $\mathsf{Manifest_{I_{dev}}}$ using $\mathsf{pk_{M_{svr}}}$, then confirming that $\mathsf{pk_{I_{dev}}}$from $\mathsf{Msg_{rep}}$ matches $\mathsf{pk_{I_{dev}}}$in $\mathsf{Manifest_{I_{dev}}}$, and finally verifying the signature on $\mathsf{Msg_{rep}}$ using $\mathsf{pk_{I_{dev}}}$from the manifest. After successful validation, $\mathsf{U_{dev}}$ configures its UWB module with the decrypted ranging parameters.

Secure Ranging: $\mathsf{U_{dev}}$ initiates STS-based SS-TWR ranging with $\mathsf{I_{dev}}$ using the configured parameters. Upon successful ranging, $\mathsf{U_{dev}}$ displays device information, and location to the user.